

Shared Libraries

1 Core Idea

Definition

Shared libraries are libraries whose code pages can be mapped into multiple processes so that only one physical copy needs to be kept in memory.

On Windows, shared libraries are commonly called **DLLs** or **Dynamic Link Libraries**.

Main Benefit

Shared libraries reduce executable size, disk usage, and RAM usage by avoiding duplicate copies of common library code.

2 Sharing Program Text

If the same program is started twice, most operating systems share its program-text pages automatically.

Page type	Sharing behavior
Text pages	Usually read-only, so they are easy to share.
Data pages	May be private, or shared read-only until modified using copy on write.

Rule

Read-only code can be safely shared. Writable data needs protection or copy on write.

3 Traditional Static Linking

In traditional linking, object files and library functions are copied into the final executable.

```
ld *.o -lc -lm
```

This command links:

- all object files in the current directory;
- the C library;
- the math library.

Undefined External

An **undefined external** is a function or symbol used by a program but not defined in its object files. The linker searches libraries to find it.

Example: if the program calls `printf`, the linker finds `printf` in a library and includes it in the executable. If `printf` calls `write`, then `write` must also be included.

4 Problem with Static Linking

Large modern programs use many libraries, such as graphics, I/O, and user-interface libraries.

Static Linking Waste

If every executable contains its own private copy of common library functions, disk space and RAM are wasted.

For example, if hundreds of programs statically include the same 20-MB to 50-MB graphics library, the same code appears many times on disk and may also appear many times in RAM.

5 How Shared Libraries Work

With shared libraries, the linker does not copy the whole library code into each executable.

Instead, it places small pieces of code called **stubs**.

Stub

A **stub** is a small routine inserted by the linker that connects the program to the real library function at run time.

Depending on the system, a shared library may be loaded:

- when the program starts; or
- when a function from the library is first called.

Paging Detail

The entire shared library is not loaded into RAM at once. It is paged in as needed, so unused functions do not need to occupy memory.

6 Update Advantage

Shared libraries also simplify bug fixes and security updates.

Static library	Shared library
Programs may need recompilation or relinking to get fixes.	Replacing the shared library can update all programs that use it.
Library code is copied into each executable.	Executables bind to the shared copy at run time.

This is important for commercial software where users do not have the source code.

7 Shared Library Address Problem

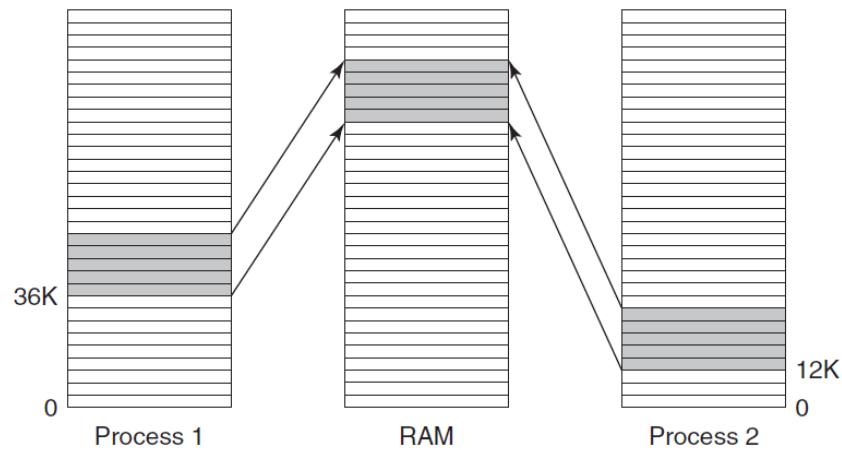


Figure 3-26. A shared library being used by two processes.

Figure Setup

The same 20-KB shared library is mapped into two processes, but at different virtual addresses: 36K in process 1 and 12K in process 2.

The physical RAM location does not matter to the program because the MMU translates virtual addresses to physical frames. The problem is the library's own internal addresses.

8 Why Absolute Addresses Fail

Suppose the first function in the library must jump to byte 16 inside the library.

Process	Correct jump target
Process 1	$36K + 16$.
Process 2	$12K + 16$.

If the library contained an absolute jump address fixed for process 1, it would be wrong for process 2.

Problem

Relocating a shared library separately for each process would create private modified copies of the library pages, defeating the purpose of sharing.

9 Position-Independent Code

Definition

Position-independent code (PIC) is code that uses relative addressing instead of absolute addresses, so it can run correctly no matter where it is mapped.

Instead of saying:

jump to absolute address X

PIC uses instructions like:

jump forward or backward by n bytes

This works because the relative distance between instructions inside the library is the same in every process.

Solution

Shared libraries are compiled to avoid absolute addresses. This lets the same physical code pages be mapped at different virtual addresses in different processes.

10 Final Summary

One-Box Summary

Shared libraries let processes share common library code.
Static linking copies library code into each executable.
Shared linking uses stubs and binds to library code at run time.
Libraries may be mapped at different virtual addresses in different processes.
Position-independent code solves this by using relative addressing.